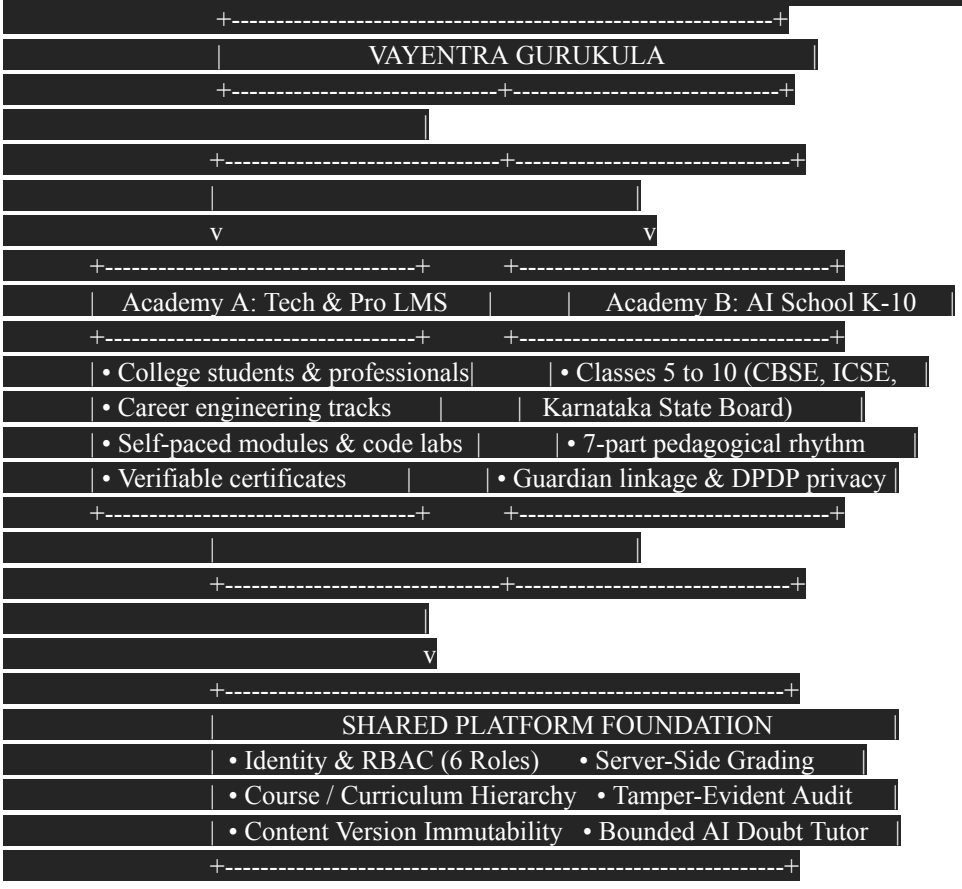
Vayentra Gurukula Learning Platform — Comprehensive Project Walkthrough

Welcome to the architectural and functional walkthrough of **Vayentra Gurukula**. This document breaks down the platform's vision, system architecture, database design, backend services, frontend user interface, and workflow mechanics.

---

1. Executive Summary & Vision

Vayentra Gurukula is a dual-academy educational platform uniting **traditional Indian pedagogical rigor** (the Gurukula methodology) with **modern AI intelligence** and **enterprise software design**.
The platform is engineered around two distinct educational universes powered by a unified core engine:



---

2. Technology Stack & Directory Structure

Tech Stack Breakdown

Layer	Technologies	Key Responsibilities

Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS v4, Lucide Icons	Responsive UI, Academy Switcher, 7-Part Episode Player, Faculty Studio, Guardian Center
Backend API	FastAPI, Pydantic v2, Python 3.12	Async REST API, RBAC, Server-Side Grading, AI Guardrails, Anti-Cheating Filters
Database & ORM	SQLAlchemy 2.0 (Async), Alembic, SQLite (aiosqlite) / PostgreSQL-ready	68 Relational Entities across 10 functional domains, foreign key cascades, unique indexes
Security & Auth	Passlib, Bcrypt, PyJWT	Constant-time password hashing, Bearer JWT session auth, guardian-child authorization scopes
Testing	Pytest, Asyncio	23 automated tests covering auth, schema invariants, content gates, grading, and RAG tutor

Workspace Directory Layout

```
vayentra gurukula web app/
├── backend/
│   ├── app/
│   │   ├── api/v1/
│   │   │   ├── endpoints/
│   │   │   │   ├── academies.py      # Academy listing and switching
│   │   │   │   ├── auth.py           # Registration, login, profile (/me)
│   │   │   │   ├── certifications.py # Certificates & verifiable credentials
│   │   │   │   ├── curricula.py      # Boards, Grades, Subjects, Chapter tree
│   │   │   │   ├── episodes.py       # 7-Part episode content & checkpoint tests
│   │   │   │   ├── faculty.py        # Faculty Studio, student telemetry, authoring
│   │   │   │   ├── health.py         # System health probe
│   │   │   │   └── tutor.py          # Bounded Socratic AI Doubt Tutor
│   │   │   └── router.py             # Master API router
│   │   ├── core/
│   │   │   ├── config.py             # Settings, JWT secrets, CORS
│   │   │   ├── database.py           # Async engine, sessionmaker, Base model
│   │   │   ├── dependencies.py       # Current user, DB session injection
│   │   │   └── security.py           # Password hashing & JWT token generators
│   │   ├── models/                   # 68 relational SQLAlchemy 2.0 models
│   │   │   ├── academy.py            # Courses, modules, lessons, enrollments
│   │   │   ├── ai.py                 # Prompts, AI jobs, token accounting
│   │   │   ├── assessment.py         # Tests, question bank, attempts, rubrics
│   │   │   ├── audit.py              # Tamper-evident immutable audit logs
```

```
│   │   │   ├── certification.py      # Course & episode credentials
│   │   │   ├── commerce.py           # Subscriptions, products, transactions
│   │   │   ├── content.py            # Episodes, versions, media, subtitles
│   │   │   ├── curriculum.py         # Boards, grades, subjects, chapters, concepts
│   │   │   ├── identity.py           # Users, roles, permissions, guardians, students
│   │   │   └── operations.py         # Classes, assignments, tickets
│   │   ├── schemas/                  # Pydantic v2 validation contracts
│   │   ├── services/                 # Business logic, mastery engine, seeder
│   │   └── main.py                   # FastAPI initialization & seed defaults
│   ├── alembic/                      # Database migration revisions
│   └── tests/                        # 23 automated pytest test suites
├── docs/                             # Canonical architecture, requirements & specs
│   ├── ARCHITECTURE.md
│   ├── DATABASE_SCHEMA.md
│   ├── REQUIREMENTS.md
│   ├── API_SPECIFICATION.md
│   ├── SECURITY_PLAN.md
│   └── PROGRESS.md
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── globals.css           # Design tokens, glassmorphism, glow utilities
│   │   │   ├── layout.tsx            # HTML root, Google Fonts (Plus Jakarta Sans)
│   │   │   └── page.tsx              # Main dashboard orchestration
│   │   └── components/
│   │       ├── Navbar.tsx            # Dual academy switch, role switch, streaks
│   │       ├── ChapterRoadmap.tsx    # Board/grade filters, chapter map, episode unlocking
│   │       ├── EpisodePlayer.tsx     # The 7-step pedagogical player & AI tutor drawer
│   │       ├── FacultyStudioView.tsx # Student roster, live attempts, episode authoring
│   │       ├── GuardianDashboard.tsx # Parent portal, linked child telemetry, DPDP privacy
│   │       ├── ReviewerPortal.tsx    # 5-Pillar academic gatekeeper checklist
│   │       └── TechAcademyView.tsx   # Technical & career engineering course catalog
│   ├── package.json
│   └── next.config.ts
└── vayentra.db                       # Pre-seeded SQLite database
```

---

3. Database Architecture (The 10 Domain Groups)

The database schema defined in
backend/app/models contains **68 relational entities** grouped into 10 domains:

```
+-------------------------------------------------------------------------------------------------+
|                                THE 10 PERSISTENCE DOMAINS                                       |
+-------------------------------------------------------------------------------------------------+
| 1. Identity & RBAC      : users, roles, permissions, user_roles, role_permissions, sessions     |
| 2. Academy LMS          : academies, courses, modules, lessons, enrollments, lesson_progress    |
| 3. Curriculum Catalogue : boards, states, grades, academic_years, media, subjects, curricula, chapters |
| 4. Content & Media      : episodes, episode_versions, media_assets, subtitles, lesson_sources   |
| 5. Assessment & Mastery : question_bank, question_versions, tests, attempts, rubrics, topic_mastery |
| 6. Family & Privacy     : guardian_profiles, student_profiles, guardian_links, consent_records  |
| 7. Commerce & Billing   : products, plans, subscriptions, entitlements, payment_transactions, refunds |
| 8. Audit Logging        : audit_events (tamper-evident immutable operations)                    |
```

```
| 9. AI Operations        : ai_jobs, prompt_templates, generation_versions, model_usage, moderation_events|
| 10. Operations          : organizations, memberships, class_groups, assignments, notifications          |
+-----------------------------------------------------------------------------------------------------+
```

Critical Database Invariants

1. **Curriculum Identity Constraint:** Unique composite key (board_id, grade_id, academic_year_id, medium_id, subject_id, version_label).
2. **Version Immutability:** When content is updated, a new
3. EpisodeVersion or
4. QuestionVersion is appended. Existing historical test attempts are permanently pinned to the version they were tested on.
5. **Publication Gate:** An episode **cannot** be published unless it has at least one associated
6. Test with 3 or more questions.
7. **Minor Integer Currency:** All payment and subscription prices are stored as non-negative integers in minor units (e.g., INR paise) to eliminate floating-point rounding errors.

---

4. Current Frontend UI & UX Architecture

The frontend is built as a single-page interactive app orchestrated in frontend/src/app/page.tsx.

1. Universal Top Navigation (Navbar.tsx)

- **Brand Identity:** Vayentra Gurukula logo with the tagline *"Ancient Wisdom × Modern AI"*.
- **Academy Switcher:** Toggle between **AI School Academy (K-10)** and **Tech & Pro Academy (Careers)**.
- **Gamification HUD:** Shows **5-Day Study Streak** and **Karma Points**.
- **Role Persona Switcher:** Allows toggling between personas for demo and testing:
  - student (Class 8 CBSE Learner)
  - faculty (Faculty Member / Founder)
  - guardian (Parent of Aarav)
  - tech_learner (Full-Stack Engineer)
  - reviewer (Academic Quality Gatekeeper)

2. School Curriculum Roadmap (ChapterRoadmap.tsx)

- **Interactive Selectors:** Filter by Board (CBSE, Karnataka State, CISCE) and Grade (Class 5 to Class 10).
- **Chapter & Episode Tree:** Shows chapter milestones with ordered episodes.
- **Progression Logic:** Completed episodes unlock the next episode in sequential order.

3. The 7-Part Pedagogical Episode Player (EpisodePlayer.tsx)

Every school lesson adheres to the 7-step pedagogical rhythm:

1. **Hook:** Everyday curiosity-provoking question.
2. **Explanation:** Deep, age-appropriate conceptual breakdown with video/audio.

3. **Worked Example:** Checked step-by-step mathematical or scientific solution.
4. **Try It:** Immediate application challenge for the student.
5. **Quick Recap:** Key formulas, definitions, and common misconceptions.
6. **Episode Checkpoint:** 3 to 5 compulsory MCQs.
    - *Security Feature:* Correct answer keys are never sent to the browser.
    - Answers are submitted to the backend via POST /api/v1/episodes/{id}/checkpoint/evaluate.
    - Passing threshold (default 60%) unlocks the next lesson.
7. **Next Step / Remediation:** Direct guidance based on whether the student mastered the concept or needs targeted review.

- **Embedded Guru AI Drawer:** Students can ask questions to the AI tutor without leaving the player.

4. Faculty & Founder Studio (FacultyStudioView.tsx)

A 41KB staff studio featuring:

- **Student Roster & Live Analytics:** View all enrolled students, attempt counts, and average test scores.
- **Manual Student Enrollment:** Directly register new students into the database.
- **Episode Authoring Suite:** Create complete 7-part lessons and checkpoint tests.
- **Course Manager:** Build technical engineering courses for Academy A.

5. Guardian Command Center (GuardianDashboard.tsx)

- Scoped to linked minor children with **DPDP Act (India) & COPPA (US)** data minimization compliance.
- Visualizes learning hours, test scores, subject mastery radar, and active subscriptions.

6. Academic Reviewer Desk (ReviewerPortal.tsx)

- Enforces the **5-Pillar Quality Gate**:
    1. Curriculum Alignment
    2. 7-Part Pedagogy Completeness
    3. Checkpoint Test Gate (≥3 questions with server keys)
    4. Accessibility & Subtitles
    5. Copyright Clearance
- Allows reviewers to Approve, Request Changes, or Reject content drafts.

7. Technical Academy View (TechAcademyView.tsx)

- Displays engineering tracks (Full-Stack, Distributed Systems, AI Engineering) with self-paced module enrollment.

---

5. Backend REST API Endpoints Overview

All endpoints are mounted under /api/v1:

Router	Method & Route	Purpose	Key Security / Business Logic
auth.py	POST /auth/register POST /auth/login GET /auth/me	User onboarding, JWT session token generation, role attachment	Bcrypt password hashing, constant-time validation
curricula.py	GET /curricula GET /curricula/{id}	Filterable catalogue of boards, grades, subjects, chapters, and episodes	Direct database join queries with async selectinload
episodes.py	GET /episodes/{id} GET /episodes/{id}/checkpoint POST /episodes/{id}/checkpoint/evaluate	Fetch 7-part lesson content, get checkpoint questions, submit test attempt	**Answer keys strictly excluded** from GET. Server calculates score and updates topic mastery
tutor.py	POST /tutor/ask	Context-bounded Socratic AI Doubt Tutor	**Anti-Cheating Guardrail:** Rejects test queries. Bounded to current lesson transcript
faculty.py	GET /faculty/students POST /faculty/students POST /faculty/episodes GET /faculty/courses	Faculty administration, manual student onboarding, content authoring	Computes aggregate test percentages from actual database test attempts

certifications.py	GET /certifications/my-certificates GET /certifications/verify/{hash}	Issue and verify digital certificates	Cryptographic SHA-256 verification hash
health.py	GET /health	System liveness probe	Database ping check

---

6. Current Verified Status & Test Suite

The backend has **23 passing automated tests** verifying core logic:

- test_auth.py: User registration, login, profile fetch.
- test_rbac.py: Role enforcement and permission boundaries.
- test_database_schema_acceptance.py: 12 acceptance criteria tests covering schema integrity, curriculum identity, version immutability, server-side grading, webhook idempotency, and query benchmarks.
- test_faculty_and_content_authoring.py: Episode authoring with checkpoint requirements and student creation.
- test_school_learning_engine.py: 7-part episode playback, answer scoring, and AI tutor guardrail refusals.

The frontend compiles cleanly with **Next.js 16 + React 19** with zero TypeScript or build errors.

---

7. Ready for Your Custom Transformation

Now that you have the complete map of the project, we can proceed with your vision to:

1. **Completely overhaul the User Interface (UI) & User Experience (UX)**.
2. **Re-architect and customize the platform's functionalities**.

To tailor this to your exact specifications, please share your preferences:

Key Areas to Define:

1. **UI / UX Style & Aesthetic:**
   - What visual theme do you want? (e.g., Ultra-modern dark glassmorphism, clean high-contrast minimal light mode, gamified vibrant educational UI, luxury academic dashboard, or something else?)
   - Do you want a **multi-page App Router architecture** (e.g., /dashboard, /learn/[episodeId], /faculty, /parent, /tech) with separate dedicated URLs instead of the current tab-swapping single-page setup?
   - How should the sidebar, header, and primary navigation be structured?
2. **Core Functionalities & Workflows:**
   - What specific features do you want to add, remove, or modify?
   - How should students, teachers, parents, and admins interact?
   - Do you want live streaming/interactive video players, gamified quiz battles, assignment submissions, code execution sandboxes, or WhatsApp/SMS parent notifications?